1. Linear Regression using Normal Form

What is the Normal Equation?

The **Normal Equation** is a closed-form (analytical) solution to linear regression. Instead of iterating, it computes the optimal weights **w** directly in a single step.

The Formula

Given:

- **X**: feature matrix of shape *(m × n)* — m samples, n features
- **y**: target vector of shape *(m,)*

$$w = (X^T X)^{-1} X^T y$$

Step-by-step breakdown

Step	Expression	Shape	What it does
1	$X^T X$	(n × n)	Gram matrix — captures feature covariance
2	$(X^T X)^{-1}$	(n × n)	Inverse of the Gram matrix
3	$X^T y$	(n,)	Projects targets onto feature space
4	$(X^T X)^{-1} X^T y$	(n,)	Optimal weights **w**

When to use it

Pros	Cons
No learning rate to tune	Matrix inversion is O(n³) — slow for large n
Exact solution in one step	Fails if $X^T X$ is singular (non-invertible)
No convergence issues	Not suitable for very high-dimensional data

> **Note:** X must include a column of ones if an intercept (bias) term is needed.

In [1]:
```
"""
Write a Python function that performs linear regression using the normal equ
and return the coefficients of the linear regression model. Round your answe
"""
```

Out[1]:
```
'\nWrite a Python function that performs linear regression using the normal
equation. The function should take a matrix X (features) and a vector y (ta
rget) as input,\nand return the coefficients of the linear regression mode
l. Round your answer to four decimal places, -0.0 is a valid result for rou
nding a very small number.\n'
```

In [23]:
```python
import numpy as np

class LinearRegressionNormal:
    def __init__(self):
        self.theta = None

    def fit(self, X: list[list[float]], y: list[float]):
        X = np.array(X)
        y = np.array(y)
        self.theta = np.linalg.inv(X.T @ X) @ X.T @ y
        return self

    def predict(self, X: list[list[float]]) -> np.ndarray:
        X = np.array(X)
        return np.round(X @ self.theta, 4)
```

Implementation Walkthrough

`self.theta = np.linalg.inv(X.T @ X) @ X.T @ y`

- `X.T @ X` — Gram matrix (n × n); `@` is NumPy's matrix multiply operator
- `np.linalg.inv(...)` — computes the exact matrix inverse
- `@ X.T @ y` — completes the normal equation, giving the optimal weights **w**
- `predict` rounds to 4 decimal places as required by the problem spec
- The first column of ones in X (e.g. `[1, x]`) acts as the bias — the corresponding weight is the intercept

In [24]:
```python
# Test Case 1
model = LinearRegressionNormal()
model.fit([[1, 1], [1, 2], [1, 3]], [1, 2, 3])
print("Coefficients:", np.round(model.theta, 4).tolist())
print("Predictions:", model.predict([[1, 1], [1, 2], [1, 3]]))

# Expected Coefficients: [0.0, 1.0]
```

```
Coefficients: [-0.0, 1.0]
Predictions: [1. 2. 3.]
```

In [25]:
```python
# Test Case 2
model = LinearRegressionNormal()
model.fit([[1, 3, 4], [1, 2, 5], [1, 3, 2]], [1, 2, 1])
print("Coefficients:", np.round(model.theta, 4).tolist())
print("Predictions:", model.predict([[1, 3, 4], [1, 2, 5], [1, 3, 2]]))

# Expected Coefficients: [4.0, -1.0, 0.0]
```

```
Coefficients: [4.0, -1.0, -0.0]
Predictions: [1. 2. 1.]
```

2. Linear Regression using Gradient Descent

What is Gradient Descent?

Gradient Descent is an iterative optimization algorithm. Instead of solving for **w** analytically, it starts from w = 0 and repeatedly nudges the weights in the direction that reduces the loss.

Loss Function: Mean Squared Error (MSE)

$$J(w) = \frac{1}{2m} \sum_{i=1}^{m} (\hat{y}^{(i)} - y^{(i)})^2$$

where $\hat{y} = Xw$ is the model's prediction. The $\frac{1}{2}$ factor is a convenience — it cancels the 2 that appears when differentiating the square.

The Gradient

Differentiating J with respect to each weight w_j:

$$\frac{\partial J}{\partial w_j} = \frac{1}{m} \sum_{i=1}^{m} (\hat{y}^{(i)} - y^{(i)}) \cdot x_j^{(i)}$$

In words: *the average of (prediction error × feature value) across all samples.*

The Update Rule

$$w_j \leftarrow w_j - \alpha \cdot \frac{\partial J}{\partial w_j}$$

All weights are updated simultaneously after computing the full-batch gradient.

Key Hyperparameters

Hyperparameter	Too small	Too large
α (learning rate)	Converges very slowly	Diverges / overshoots
iterations	Underfits	Wastes compute (but won't hurt if α is right)

3 Ways to Implement It

Below we implement gradient descent three times, progressively eliminating explicit Python loops using NumPy vectorization:

In [26]:
```
"""
Write a Python function that performs linear regression using gradient desce
and y (target) as input, along with the learning rate alpha and the number c

Requirements
Minimize the Mean Squared Error (MSE) loss function:

m is the number of samples. The factor of 1/2 simplifies the gradient calcul

Initialize all weights to zero

Use batch gradient descent (use all samples in each iteration)

The input matrix X has shape (m, n) where m is the number of training exampl
The target vector y has shape (m,).
"""
```

Out[26]:
```
'\nWrite a Python function that performs linear regression using gradient d
escent. The function should take NumPy arrays X (features with a column of
ones for the intercept) \nand y (target) as input, along with the learning
rate alpha and the number of iterations. Return the learned coefficients (w
eights) as a NumPy array.\n\nRequirements\nMinimize the Mean Squared Error
(MSE) loss function: \n\nm is the number of samples. The factor of 1/2 simp
lifies the gradient calculation.\n\nInitialize all weights to zero\n\nUse b
atch gradient descent (use all samples in each iteration)\n\nThe input matr
ix X has shape (m, n) where m is the number of training examples and n is t
he number of features (including the bias column of ones). \nThe target vec
tor y has shape (m,).\n'
```

Version	Loops removed	Core idea
3-Loop	None	Fully explicit: iterate over epochs → features → samples
2-Loop	Sample loop	Use `np.sum` to aggregate over all m samples at once
1-Loop	Sample + feature loops	One matrix multiply updates all w_j simultaneously

---

Version 1 — 3-Loop (Most Explicit)

In [27]:
```
# using 3 loop implementation
class LinearRegressionGD3Loop:
    def __init__(self, alpha: float = 0.01, iterations: int = 1000):
```

```
        self.alpha = alpha
        self.iterations = iterations
        self.theta = None

    def fit(self, X: np.ndarray, y: np.ndarray):
        m, n = X.shape
        y = y.reshape(-1, 1)
        self.theta = np.zeros((n, 1))

        for _ in range(self.iterations):
            for j in range(n):
                gradient_sum = 0
                for i in range(m):
                    gradient_sum += (X[i] @ self.theta - y[i]) * X[i, j]
                gradient = (1 / m) * gradient_sum
                self.theta[j] = self.theta[j] - self.alpha * gradient
        return self

    def predict(self, X: np.ndarray) -> np.ndarray:
        return (X @ self.theta).flatten()
```

3-Loop Walkthrough

```
for each epoch:                          # Loop 1 — repeat
until convergence
    for j in range(n):                   # Loop 2 — one weight
w_j at a time
        grad_sum = 0
        for i in range(m):               # Loop 3 — accumulate
over each sample
            ŷ_i = X_i · w                # prediction for sample
i
            grad_sum += (ŷ_i − y_i) * X_ij
        w_j −= α * (grad_sum / m)
```

- **Most readable** — mirrors the math formula exactly, term by term
- **Slowest** — three nested Python loops: O(iterations × n × m) overhead
- `X[i] @ w` computes $\hat{y}_i$ for the i-th sample (dot product of a row with w)
- Each weight's gradient is accumulated sample-by-sample before the update

In [28]:
```
# Test Case 1
model = LinearRegressionGD3Loop(alpha=0.1, iterations=1000)
model.fit(np.array([[1, 1], [1, 2], [1, 3]]), np.array([3, 5, 7]))
print("Coefficients:", np.round(model.theta.flatten(), 4))
print("Predictions:", np.round(model.predict(np.array([[1, 1], [1, 2], [1, 3
# Expected Coefficients: [1. 2.]

# Test Case 2
model = LinearRegressionGD3Loop(alpha=0.05, iterations=1000)
model.fit(np.array([[1, 2], [1, 3], [1, 4], [1, 5]]), np.array([7, 9, 11, 13
print("Coefficients:", np.round(model.theta.flatten(), 4))
# Expected Coefficients: [2.9699 2.0078]
```

```
Coefficients: [1. 2.]
Predictions: [3. 5. 7.]
Coefficients: [2.9756 2.0063]
```

Version 2 — 2-Loop (Vectorize Over Samples)

In [29]:
```python
# using 2 loop implementation
class LinearRegressionGD2Loop:
    def __init__(self, alpha: float = 0.01, iterations: int = 1000):
        self.alpha = alpha
        self.iterations = iterations
        self.theta = None

    def fit(self, X: np.ndarray, y: np.ndarray):
        m, n = X.shape
        y = y.reshape(-1, 1)
        self.theta = np.zeros((n, 1))

        for _ in range(self.iterations):
            for j in range(n):
                gradient = (1 / m) * np.sum((X @ self.theta - y) * X[:, j].r
                self.theta[j] = self.theta[j] - self.alpha * gradient
        return self

    def predict(self, X: np.ndarray) -> np.ndarray:
        return (X @ self.theta).flatten()
```

2-Loop Walkthrough

```
for each epoch:                          # Loop 1 — still
iterating over epochs
    for j in range(n):                   # Loop 2 — still one
weight at a time
        ŷ   = X @ w                     # shape (m, 1) — all m
predictions at once
        grad = (1/m) * np.sum((ŷ - y) * X[:, j])
        wⱼ  -= α * grad
```

- **Sample loop eliminated**: `np.sum(...)` replaces the explicit sample-wise accumulation
- `X @ w` computes all m predictions ŷ in one matrix multiply
- `X[:, j]` selects the j-th feature column across all m samples
- Still loops over features — partially vectorized

In [30]:
```python
# Test Case 1
model = LinearRegressionGD2Loop(alpha=0.1, iterations=1000)
model.fit(np.array([[1, 1], [1, 2], [1, 3]]), np.array([3, 5, 7]))
print("Coefficients:", np.round(model.theta.flatten(), 4))
print("Predictions:", np.round(model.predict(np.array([[1, 1], [1, 2], [1, 3
# Expected Coefficients: [1. 2.]
```

```
# Test Case 2
model = LinearRegressionGD2Loop(alpha=0.05, iterations=1000)
model.fit(np.array([[1, 2], [1, 3], [1, 4], [1, 5]]), np.array([7, 9, 11, 13
print("Coefficients:", np.round(model.theta.flatten(), 4))
# Expected Coefficients: [2.9699 2.0078]
```

```
Coefficients: [1. 2.]
Predictions: [3. 5. 7.]
Coefficients: [2.9756 2.0063]
```

Version 3 — 1-Loop (Fully Vectorized)

In [31]:
```
# using 1 loop (fully vectorized) implementation
class LinearRegressionGD:
    def __init__(self, alpha: float = 0.01, iterations: int = 1000):
        self.alpha = alpha
        self.iterations = iterations
        self.theta = None

    def fit(self, X: np.ndarray, y: np.ndarray):
        m, n = X.shape
        y = y.reshape(-1, 1)
        self.theta = np.zeros((n, 1))

        for _ in range(self.iterations):
            gradient = (1 / m) * X.T @ (X @ self.theta - y)
            self.theta = self.theta - self.alpha * gradient
        return self

    def predict(self, X: np.ndarray) -> np.ndarray:
        return (X @ self.theta).flatten()
```

1-Loop (Fully Vectorized) Walkthrough

```
for each epoch:                          # The only loop
    ŷ   = X @ w                         # shape (m, 1) — all
predictions
    grad = (1/m) * Xᵀ @ (ŷ - y)        # shape (n, 1) —
gradient for ALL w at once
    w   -= α * grad                      # update ALL weights
simultaneously
```

- `X @ w` — shape *(m, 1)*: ŷ (predictions) for every sample in one op
- `ŷ - y` — shape *(m, 1)*: prediction errors for all samples
- `X.T @ (ŷ - y)` — shape *(n, 1)*: computes $\sum_i (\hat{y}^{(i)} - y^{(i)}) \cdot x_j^{(i)}$ for **all j at once** — both the sample loop and feature loop are gone
- **Fastest in practice** — NumPy delegates to optimized BLAS/LAPACK routines

---

Performance Summary

Version	Python Loops	Dominant cost	Relative speed
3-Loop	iterations × n × m	Interpreted Python	Slowest
2-Loop	iterations × n	Partial NumPy ops	Medium
1-Loop	**iterations**	**Pure NumPy (BLAS)**	**Fastest**

> **Takeaway:** Always prefer the fully vectorized form. The 1-loop version is what you would use in production code.

In [32]:
```
# Test Case 1
model = LinearRegressionGD(alpha=0.1, iterations=1000)
model.fit(np.array([[1, 1], [1, 2], [1, 3]]), np.array([3, 5, 7]))
print("Coefficients:", np.round(model.theta.flatten(), 4))
print("Predictions:", np.round(model.predict(np.array([[1, 1], [1, 2], [1, 3
# Expected Coefficients: [1. 2.]

# Test Case 2
model = LinearRegressionGD(alpha=0.05, iterations=1000)
model.fit(np.array([[1, 2], [1, 3], [1, 4], [1, 5]]), np.array([7, 9, 11, 13
print("Coefficients:", np.round(model.theta.flatten(), 4))
# Expected Coefficients: [2.9699 2.0078]
```

```
Coefficients: [1. 2.]
Predictions: [3. 5. 7.]
Coefficients: [2.9699 2.0078]
```